Number Theory

Elijah Renner

October 7, 2024

Contents

1 Introduction

Number theory studies the set of natural numbers $\mathbb{N} = \{1, 2, 3, 4, 5, 6, 7, ...\}$.

$1 + 2 + 3 + ... + (n-1) + n = \frac{n(n+1)}{2}$.

Square numbers $1, 4, 9, 16, 25, 36, ...$ are numbers that can be organized in squares.

Triangular numbers $1, 3, 6, 10, 15, 21, ...$ are numbers that can be arranged in a triangle.

Prime numbers $2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, ...$ are numbers whose divisors are only themselves and 1. Composite numbers are $\mathbb{N}$ with the primes removed.

Perfect numbers $6, 28, 496, ...$ are numbers whose divisors sum to the perfect number.

Fibonacci numbers $1, 1, 2, 3, 5, 8, 13, 21, 34, ...$ are created by starting with 1 and 1. Then, each number in the list the sum of the previous two.

2 Big O Notation

Formally, we say that a function $f(n)$ is $O(g(n))$ if there exist positive constants c and n_0 such that:

$$f(n) \leq c \cdot g(n) \quad \text{for all } n \geq n_0.$$

Here: - $f(n)$ represents the actual runtime or space complexity of the algorithm. - $g(n)$ is a comparison function that represents the upper bound. - c and n_0 are constants.

Common Big O Notation Examples

Here are some commonly used Big O notations:

- $O(1)$ Constant Time: The runtime does not depend on the size of the input.
- $O(\log n)$ Logarithmic Time: The runtime grows logarithmically with the input size.
- $O(n)$ Linear Time: The runtime grows linearly with the input size.
- $O(n \log n)$ Quasi-Linear Time: The runtime grows at a rate proportional to $n \log n$.
- $O(n^2)$ Quadratic Time: The runtime grows quadratically with the input size.
- $O(2^n)$ Exponential Time: The runtime doubles with each additional unit increase in input size.
- $O(n!)$ Factorial Time: The runtime grows factorially with the input size.

Example Analysis

Consider the function $f(n) = 3n^2 + 5n + 2$. To determine its Big O notation:

1. Identify the dominant term: In this case, $3n^2$. 2. Drop the constant coefficients: The Big O notation becomes $O(n^2)$.

Thus, $f(n) = O(n^2)$.

Properties of Big O Notation

Big O notation has some useful mathematical properties:

- **Transitivity:** If $f(n) = O(g(n))$ and $g(n) = O(h(n))$, then $f(n) = O(h(n))$.
- **Addition:** If $f_1(n) = O(g_1(n))$ and $f_2(n) = O(g_2(n))$, then $f_1(n) + f_2(n) = O(\max(g_1(n), g_2(n)))$.
- **Multiplication by a Constant:** If $f(n) = O(g(n))$, then $c \cdot f(n) = O(g(n))$ for any constant $c > 0$.